

MAUMart — Deployment Readiness Assessment

CSC 720 marketplace application (Flet client, FastAPI backend) · Assessed 1 October 2026

1. Verdict

Not ready for deployment to real users, and not efficient at realistic scale. The application is a functionally complete, well-tested prototype. Measurement found three defects that make real use unsafe: it oversells stock and charges every buyer; the recommendation endpoint takes about 7 seconds at 5,000 orders and stops responding under 10 concurrent users; and live payments cannot work as written. By my estimate, fixing the overselling, the recommendation endpoint and the browse endpoint is about two days of work; building real payments is another three to five. After the first group, a supervised campus pilot is reasonable. A public launch handling real money is not, for the reasons in Section 5.

2. What was measured

Single CPU core shared with the load generator, SQLite, one uvicorn worker, synthetic data (500 listings, 5,006 orders, 12,497 order items). Absolute latencies are pessimistic; the query counts and the pass/fail findings do not depend on hardware.

Test	Conditions	Result
Recommendations, seed data	15 listings, 1 request	12 ms (fine)
Recommendations, realistic data	5,006 orders, 1 request	6.8 s; 5,019 SQL queries (one per order)
Recommendations, concurrent	20 requests, 10 at a time	0 of 20 completed within 60 s
Browse /listings	500 listings, 10 concurrent	Median 2.37 s, p95 2.77 s, 4 req/s; 507 SQL queries; 210 KB, unpaginated
SSR storefront home	500 listings, 10 concurrent	Median 127 ms, 70 req/s (capped at 12 items)
Last-unit checkout	3 simultaneous buyers, 1 unit in stock, 3 trials	All 3 charged (HTTP 200) in every trial; stock ended at -1 or 0
Live payment path	Paystack's documented success response, simulated	charge() returns "pending"; checkout accepts only "success", so the order is marked failed and rejected (402)

3. Readiness by area

Area	Status	Evidence
Core transactional flow	OK	29 functional cases pass on SQLite and PostgreSQL; negative cases covered
Stock integrity under concurrency	FAIL	Overselling reproduced on SQLite; check and decrement are separate statements
Performance at realistic scale	FAIL	Recommendations and browse degrade as shown above (N+1 queries, no pagination)
Payments	FAIL	Mock only. Live mode rejects every payment; no webhook route; signature verifier tested but unused. With the mock in production, anyone can "pay" without money moving
Horizontal scaling	NOT READY	Sessions, rate limits and idempotency records are in-process; two instances would disagree
Security controls	PARTIAL	bcrypt, expiry, rate limits, CORS, headers, ORM, dependency scan. No penetration test, no admin audit log, no logout or 2FA

Notifications	UNVERIFIED	Console backend by default; SMTP verified only against a local test server
Legal and compliance	DRAFT	Four policy documents with unresolved placeholders; no legal review; no DPIA
Deployment and CI config	UNVERIFIED	Dockerfiles, Compose and CI written, never executed
Observability	PARTIAL	JSON request logs exist; no metrics, alerting or tracing
Backup and recovery	PARTIAL	Backup/restore cycle tested once; nothing scheduled or monitored
Accessibility	PARTIAL	SSR storefront passes static checks; Flet dashboards untested; no assistive-technology testing
Requirements validity	UNVERIFIED	Elicitation instruments written, never run; requirements rest on inference

4. What must change, in priority order

Effort figures are my estimates, not measurements. Each item has an acceptance test so "fixed" is something you can check.

#	Fix	Est.	Acceptance test
P0-1	Make the stock decrement atomic (UPDATE ... WHERE quantity >= n, zero rows means insufficient stock) and reserve before charging, not after	0.5 day	Race test with 3 simultaneous buyers for 1 unit: exactly 1 succeeds, 2 get 400, stock is 0, exactly 1 charge
P0-2	Precompute the item-item matrix offline (or cache per user with a TTL); remove per-request work proportional to total orders	1 day	p95 under 300 ms with 5,000 orders and 10 concurrent users
P0-3	Paginate /listings and the storefront category and search pages; eager-load photos and vendor	0.5 day	Page of 50 items in 5 queries or fewer
P1-4	Move sessions, rate-limit counters and idempotency records to Redis (or signed tokens with a denylist)	1 day	Two instances behind a balancer: login on one, authenticated request on the other
P1-5	Build the real payment flow: redirect step, pending-order state, signature-verified webhook route, verify-endpoint fallback, idempotent confirmation	3 to 5 days	Full payment completed in Paystack test mode from a network that can reach it; replayed webhook changes nothing
P1-6	Run the Docker build and the CI workflow for real; fix what breaks	0.5 day	Green CI run; docker compose up serves the app
P1-7	Legal review of the four documents; decide what email verification gates; add an admin audit log	External	Signed-off documents; verification enforced where decided
P2-8	Run the elicitation; load test on PostgreSQL with several workers; external penetration test; accessibility audit with assistive technology	Weeks	Findings recorded, requirements revised where contradicted

5. What "ready" can realistically mean

Supervised campus pilot (tens of users, one operator, you watching). Reachable after P0-1 to P0-3 (about two days by my estimate). Payments must be handled outside the app, by cash on delivery or by you confirming transfers manually, because the mock gateway marks every payment successful without moving money. Do not take real money through it.

Public launch with real money. Needs P1 as well, in particular real payments, shared session state, a real deployment run, and legal sign-off. Realistically weeks of work, not days.

Efficiency specifically. The transactional paths are fast (single recommendation request 12 ms on small data; storefront 127 ms median). The inefficiency is concentrated in two endpoints that do work proportional to total data volume on every request. That is a good position to be in, because it is a small, well-located problem.

6. Limits of this assessment

- Hardware: one core shared with the load generator, so absolute latencies understate what a real server would deliver. The 507 and 5,019 query counts are engine-independent.
- Database: all load and race tests ran on SQLite. PostgreSQL was not benchmarked, and the race on PostgreSQL is expected, not measured.
- Data: synthetic and uniformly random, so real recommendation and catalogue access patterns will differ.
- Concurrency: three threads is enough to prove the race exists; it says nothing about how often it occurs in real traffic.
- Not tested at all: login throughput under bcrypt cost (rate limited), multi-instance behaviour, real network latency, mobile builds, the Flet client under load.

7. Corrections to earlier deliverables

This assessment found errors in things I delivered earlier. All are now corrected in place:

- I said SQLite's serialised writes would mask the stock race. It did not; the race reproduces on SQLite. Corrected in the implementation Q&A (three paragraphs).
- I said going live with payments only required setting MAUMART_PAYMENT MOCK=false and supplying a key. That was false. Corrected in the payment_gateway.py docstring, the README, the course Q&A bank, the implementation Q&A and the report (new Section 12.13).
- Report Section 12.12 said nothing was left unverified where verification was possible. Concurrency and load were never tested. Section 12.13 now states this.
- I described the application tier as stateless. With in-memory sessions it is not, and the README's known-gaps list now says so.